

ASSIGNMENT 1 REPORT

ECG and PCG Analysis using CWT and STFT to Acquire Center of Gravity Area of S1 and S2



Name : Jeremia Christ Immanuel Manalu
NRP : 5023231017
Course : Non-Stationary Signal Analysis
Class : A
Lecturer : Muhammad Adib Syamlan, S.T., M.T.
Department : Biomedical Engineering

**FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS
TECHNOLOGY
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
2025**

CHAPTER I. FUNDAMENTAL THEORY

1.1. ECG Signal

The Electrocardiogram (ECG) is a recording of the electrical activity generated by the heart muscle (myocardium) during each cardiac cycle. It represents the temporal sum of action potentials generated by cardiac cells. The signal is characterized by five prominent waveforms: P, Q, R, S, and T. The QRS complex is the most distinct feature, representing the depolarization of the ventricles. Because of its high amplitude and sharp slope, the R-peak is the standard fiducial point used for time-referencing in signal processing. In the context of combined ECG-PCG analysis, the ECG acts as the "trigger" signal; the R-peak marks the beginning of a cardiac cycle, allowing for the segmentation of the associated mechanical sounds.

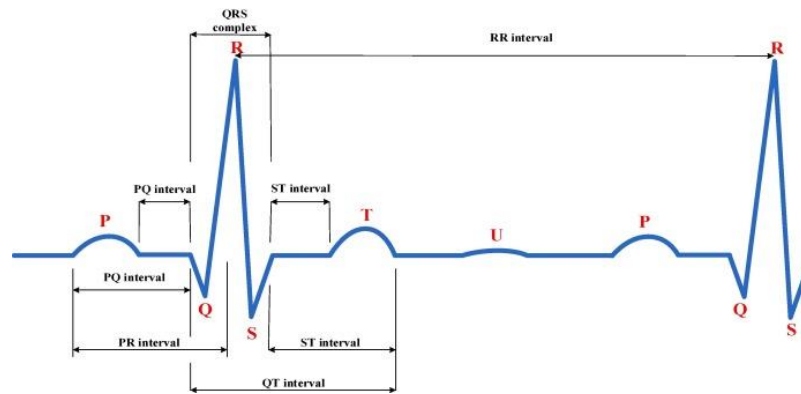


Figure 1.1. An example of ECG Signal and Its Components

1.2. PCG Signal

The Phonocardiogram (PCG) is the recording of the acoustic waves produced by the mechanical action of the heart, specifically the vibration of valves and blood flow turbulence. A normal PCG signal consists of two primary heart sounds:

- First heart sound (S1), associated with the closure of the Atrioventricular (AV) valves (mitral and tricuspid). It occurs shortly after the onset of the QRS complex in the ECG (onset of systole).
- Second Heart Sound (S2), associated with the closure of the Semilunar valves (aortic and pulmonary). It occurs near the end of the T-wave in the ECG (end of systole/onset of diastole). Analyzing PCG provides insight into the mechanical timing and frequency content of valve closures, which complements the electrical information from the ECG.

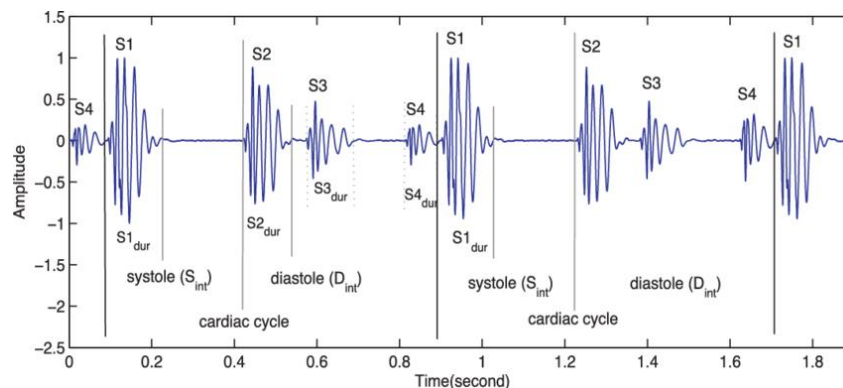


Figure 1.2. An example of PCG Signal and Its Components

1.3. Segmentation for One Cycle

To analyze heart sounds effectively, continuous signals must be segmented into individual beats. Since the electrical activation (ECG) precedes mechanical contraction (PCG), the R-R interval is the standard metric for defining a cycle. Segmentation involves identifying the index of an R-peak ($t = 0$) and defining a window that extends to the next R-peak. This process isolates the S1 and S2 sounds specific to that beat. To prevent "spectral bleeding" from adjacent cycles (e.g., the S1 of the next beat appearing at the end of the current window), the segmentation window is often clamped to the midpoint between R-peaks.

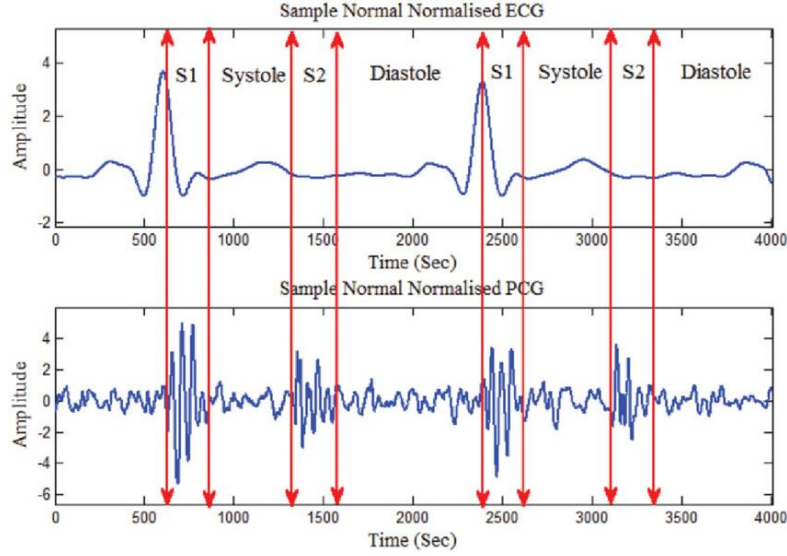


Figure 1.3. Example of A Segmentation between ECG and PCG Cycle

1.4. Fast Fourier Transform (FFT)

The Fourier Transform is a mathematical technique that decomposes a time-domain signal into its constituent sinusoidal frequencies. For digital signals, the Discrete Fourier Transform (DFT) is used. The Fast Fourier Transform (FFT) is an optimized algorithm to compute the DFT. It converts a sequence of N complex numbers $x[n]$ into another sequence of complex numbers $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}$$

While FFT provides the frequency composition of a signal, it assumes stationarity, meaning it calculates the frequency content for the *entire* duration of the signal. It loses all temporal information; looking at an FFT spectrum, one cannot determine *when* a specific frequency occurred. This limitation makes raw FFT unsuitable for analyzing non-stationary signals like PCG, where the timing of S1 and S2 is crucial.

1.5. Short Time Fourier Transform (STFT)

To overcome the loss of time information in FFT, the Short Time Fourier Transform (STFT) introduces a sliding window mechanism. The signal is divided into short segments using a window function $w(t)$ (such as a Hanning or Hamming window), and the Fourier Transform is computed for each segment. This results in a time-frequency distribution called a spectrogram. Mathematically, the STFT of a signal $x(t)$ is defined as:

$$STFT(\tau, \omega) = \int_{-\infty}^{\infty} x(t)w(t - \tau)e^{-j\omega t}dt$$

where τ represents the time shift of the window and ω is the frequency. The STFT is subject to the Heisenberg Uncertainty Principle (Gabor limit): a short window provides good time resolution but poor frequency resolution, while a long window provides good frequency resolution but poor time resolution. This trade-off is fixed for the entire analysis.

1.6. Continuous Wavelet Transform (CWT)

The Continuous Wavelet Transform (CWT) offers a solution to the fixed resolution of STFT by using a multi-resolution analysis approach. Instead of a fixed window, CWT correlates the signal with a "Mother Wavelet" function $\psi(t)$ that is scaled (dilated or compressed) and translated (shifted in time). The CWT is defined as:

$$CWT(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t)\psi^*\left(\frac{t-b}{a}\right)dt$$

where a is the scale factor (inversely proportional to frequency), b is the time shift, and ψ^* denotes the complex conjugate.

- Small scales (a) corresponds to high frequencies. The wavelet is compressed, providing high time resolution (ideal for detecting the sharp start of S1/S2).
- Large scales (a) corresponds to low frequencies. The wavelet is stretched, providing high frequency resolution.

For PCG analysis, the Morlet Wavelet is frequently chosen as the mother wavelet because it is essentially a Gaussian-windowed complex sinusoid, closely resembling the transient nature of heart sounds.

1.7. Center of Gravity (CoG)

After transforming the signal into a time-frequency representation (Spectrogram from STFT or Scalogram from CWT), the energy of the S1 and S2 sounds appears as distinct "blobs" or regions of high intensity. To quantify the exact location of these sounds, the Center of Gravity (CoG), or geometric centroid is calculated. This method treats the energy magnitude at each pixel as a "mass." The CoG coordinates (t_{cog}, f_{cog}) for a specific region are calculated using the first statistical moment:

$$t_{cog} = \frac{\sum_i \sum_j t_j \cdot E(f_i, t_j)}{\sum_i \sum_j E(f_i, t_j)}, f_{cog} = \frac{\sum_i \sum_j f_i \cdot E(f_i, t_j)}{\sum_i \sum_j E(f_i, t_j)}$$

where $E(f_i, t_j)$ is the energy (magnitude squared) at frequency bin i and time step j . This provides a precise, weighted average position of the heart sound in both time and frequency.

CHAPTER II. RESULT AND ANALYSIS

2.1. Problems Statement

Step:

- Pan-Tompkins Algorithm, to find R-R interval on ECG data.
- Segment based on ECG data to get PCG data (only 1 segment) based on R-R interval.
- CWT the PCG data.
- Threshold the magnitude → Plot the area → Find CoG coordinate.

2.2. Code Explanation

The software developed for this assignment operates on a sequential method designed to transform raw physiological data (PCG and ECG) into quantitative features. The process begins with the preprocessing of the ECG signal using a manual implementation of the Pan-Tompkins algorithm to detect R-peak points. These points act as time markers to segment the PCG signal into individual cardiac cycles. Once segmented, the PCG signal undergoes Time-Frequency analysis using two distinct manual methods, that are the Short-Time Fourier Transform (STFT) and the Continuous Wavelet Transform (CWT). Finally, the resulting spectrograms or scalograms are processed using image segmentation techniques (thresholding and connected-component labeling) to isolate the S1 and S2 heart sounds and calculate their Center of Gravity (CoG).

a. *Pan_Tompkins.py*

```
1. def _design_bandpass_fir(fs: int, low: float, high: float, kernel_len:
   Optional[int] = None) -> np.ndarray:
2.     if kernel_len is None:
3.         kernel_len = int(round(min(1025, max(31, 0.128 * fs))))
4.         kernel_len = int(kernel_len)
5.         if kernel_len % 2 == 0:
6.             kernel_len += 1
7.         nyq = 0.5 * fs
8.         f1 = float(low) / nyq
9.         f2 = float(high) / nyq
10.        m = (kernel_len - 1) // 2
11.        n = np.arange(-m, m+1, dtype=float)
12.        h = (np.sinc(2 * f2 * n) - np.sinc(2 * f1 * n))
13.        w = 0.5 * (1.0 - np.cos(2.0 * np.pi * (np.arange(kernel_len) /
   float(kernel_len - 1))))
14.        h *= w
15.        denom = np.sum(np.abs(h))
16.        if denom <= 0:
17.            denom = 1.0
18.        h /= denom
19.        return h
20.
```

```

21. def _five_point_derivative(sig: np.ndarray, fs: int) -> np.ndarray:
22.     kernel = np.array([-1.0, -2.0, 0.0, 2.0, 1.0])
23.     deriv = np.convolve(sig, kernel, mode='same')
24.     deriv *= (fs / 8.0)
25.     return deriv
26. def detect_r_peaks(ecg: np.ndarray,
27.                    fs: int = 2000,
28.                    low_hz: float = 5.0,
29.                    high_hz: float = 15.0,
30.                    integration_ms: float = 150.0,
31.                    refractory_ms: float = 200.0,
32.                    search_back_factor: float = 1.66,
33.                    debug: bool = False) -> np.ndarray:
34.
35.     ecg = np.asarray(ecg).ravel()
36.     N = ecg.size
37.     if N == 0:
38.         return np.array([], dtype=int)
39.
40.     # amplitude stabilization: suppress large early transients if present
41.     try:
42.         ecg_proc, changed = _amplitude_stabilize(ecg, fs, env_win_ms=200.0,
43. max_ratio_thresh=8.0, gain_clip=(0.3, 3.0))
44.         if changed and debug:
45.             print("[PT] amplitude stabilization applied (transient suppressed)")
46.     except Exception:
47.         ecg_proc = ecg.copy()
48.
49.     # 1) bandpass filter (replace butter+filtfilt)
50.     try:
51.         kernel = _design_bandpass_fir(fs, low_hz, high_hz, kernel_len=None)
52.         ecg_f = _filtfilt_like_linear_phase(ecg_proc, kernel)
53.     except Exception:
54.         ecg_f = ecg_proc.copy()
55.
56.     # 2) derivative
57.     deriv = _five_point_derivative(ecg_f, fs)
58.
59.     # 3) squaring
60.     squared = deriv ** 2
61.
62.     # 4) moving window integration
63.     mwi = _moving_window_integration(squared, fs, integration_ms)
64.
65.     # gather candidate peaks from integrated signal
66.     min_dist = max(1, int(round(refractory_ms / 1000.0 * fs)))

```

```

66. cand_idx = _find_peaks_numpy(mwi, distance=min_dist)
67.
68. if cand_idx.size == 0:
69.     if debug:
70.         print("[PT] No MWI peaks found; returning empty array")
71.     return np.array([], dtype=int)
72.
73. cand_vals_i = mwi[cand_idx]
74. cand_vals_f = np.abs(ecg_f[cand_idx])
75.
76. # Initialization using first segment
77. n_init = min(8, cand_idx.size)
78. init_idx = cand_idx[:n_init]
79. init_i = mwi[init_idx] if init_idx.size > 0 else np.array([0.0])
80. init_f = np.abs(ecg_f[init_idx]) if init_idx.size > 0 else np.array([0.0])
81.
82. SPKI = float(np.max(init_i)) if init_i.size > 0 else 1.0
83. NPKI = float(np.median(init_i) * 0.5) if init_i.size > 0 else SPKI * 0.125
84. SPKF = float(np.max(init_f)) if init_f.size > 0 else 1.0
85. NPKF = float(np.median(init_f) * 0.5) if init_f.size > 0 else SPKF *
    0.125
86.
87. THRESHOLD_I = NPKI + 0.25 * (SPKI - NPKI)
88. THRESHOLD_F = NPKF + 0.25 * (SPKF - NPKF)
89. THRESHOLD_I2 = 0.5 * THRESHOLD_I
90. THRESHOLD_F2 = 0.5 * THRESHOLD_F
91.
92. detected = []
93. detected_i_vals = []
94. detected_f_vals = []
95. rr_intervals = []
96.
97. for idx_cand in cand_idx:
98.     val_i = float(mwi[idx_cand])
99.     val_f = float(np.abs(ecg_f[idx_cand]))
100.
101.     # enforce refractory
102.     if detected and (idx_cand - detected[-1]) < min_dist:
103.         NPKI = 0.125 * val_i + 0.875 * NPKI
104.         NPKF = 0.125 * val_f + 0.875 * NPKF
105.         THRESHOLD_I = NPKI + 0.25 * (SPKI - NPKI)
106.         THRESHOLD_F = NPKF + 0.25 * (SPKF - NPKF)
107.         THRESHOLD_I2 = 0.5 * THRESHOLD_I
108.         THRESHOLD_F2 = 0.5 * THRESHOLD_F
109.         continue
110.

```

```

111.         accept = False
112.         threshold_used = None
113.
114.         if (val_i >= THRESHOLD_I) or (val_f >= THRESHOLD_F):
115.             accept = True
116.             threshold_used = 'primary'
117.         elif (val_i >= THRESHOLD_I2) or (val_f >= THRESHOLD_F2):
118.             accept = True
119.             threshold_used = 'secondary'
120.
121.         if accept:
122.             refined = _refine_to_ecg_peak(ecg, idx_cand, fs, rad_ms=30.0)
123.
124.             is_t_wave = False
125.             if detected:
126.                 prev = detected[-1]
127.                 dt = (refined - prev) / float(fs)
128.                 if dt < 0.36:
129.                     curr_slope = _max_slope_around(ecg, refined, fs)
130.                     prev_slope = _max_slope_around(ecg, prev, fs)
131.                     if prev_slope > 0 and curr_slope < 0.5 * prev_slope:
132.                         is_t_wave = True
133.
134.             if is_t_wave:
135.                 NPKI = 0.125 * val_i + 0.875 * NPKI
136.                 NPKF = 0.125 * val_f + 0.875 * NPKF
137.             else:
138.                 if not detected or (refined - detected[-1]) >= min_dist:
139.                     detected.append(int(refined))
140.                     detected_i_vals.append(val_i)
141.                     detected_f_vals.append(val_f)
142.                     if threshold_used == 'primary':
143.                         SPKI = 0.125 * val_i + 0.875 * SPKI
144.                         SPKF = 0.125 * val_f + 0.875 * SPKF
145.                     else:
146.                         SPKI = 0.25 * val_i + 0.75 * SPKI
147.                         SPKF = 0.25 * val_f + 0.75 * SPKF
148.                     if len(detected) >= 2:
149.                         rr = detected[-1] - detected[-2]
150.                         rr_intervals.append(rr)
151.                     else:
152.                         NPKI = 0.125 * val_i + 0.875 * NPKI
153.                         NPKF = 0.125 * val_f + 0.875 * NPKF
154.             else:
155.                 NPKI = 0.125 * val_i + 0.875 * NPKI
156.                 NPKF = 0.125 * val_f + 0.875 * NPKF

```



```

157.
158.     THRESHOLD_I = NPKI + 0.25 * (SPKI - NPKI)
159.     THRESHOLD_F = NPKF + 0.25 * (SPKF - NPKF)
160.     THRESHOLD_I2 = 0.5 * THRESHOLD_I
161.     THRESHOLD_F2 = 0.5 * THRESHOLD_F
162.
163.     detected = np.array(detected, dtype=int)
164.
165.     # Search-back for missed beats
166.     if detected.size >= 2:
167.         rr_ms = np.diff(detected)
168.         if rr_ms.size > 0:
169.             RRavg = int(np.mean(rr_ms[-8:]))
170.             if RRavg > 0:
171.                 RRmiss = int(round(search_back_factor * RRavg))
172.                 new_found = []
173.                 for i in range(len(detected) - 1):
174.                     a = detected[i]; b = detected[i+1]
175.                     gap = b - a
176.                     if gap > RRmiss:
177.                         lo = a + 1; hi = b - 1
178.                         if lo >= hi:
179.                             continue
180.                         local_idx_rel = int(np.argmax(mwi[lo:hi+1]))
181.                         local_idx = lo + local_idx_rel
182.                         if mwi[local_idx] >= THRESHOLD_I2:
183.                             refined = _refine_to_ecg_peak(ecg, local_idx, fs,
rad_ms=30.0)
184.                             new_found.append(int(refined))
185.                             SPKI = 0.25 * float(mwi[local_idx]) + 0.75 * SPKI
186.                             SPKF = 0.25 * float(np.abs(ecg_f[local_idx])) + 0.75
* SPKF
187.                             THRESHOLD_I = NPKI + 0.25 * (SPKI - NPKI)
188.                             THRESHOLD_F = NPKF + 0.25 * (SPKF - NPKF)
189.                 if new_found:
190.                     detected = np.unique(np.concatenate([detected,
np.array(new_found, dtype=int)]))
191.                     detected.sort()
192.
193.     # Final pass: remove too-close peaks (within refractory) keeping the
strongest ECG amplitude
194.     if detected.size > 1:
195.         final = []
196.         i = 0
197.         while i < detected.size:
198.             block = [detected[i]]

```

```

199.         j = i + 1
200.         while j < detected.size and (detected[j] - detected[i]) < min_dist:
201.             block.append(detected[j]); j += 1
202.         if len(block) == 1:
203.             final.append(block[0])
204.         else:
205.             vals = [abs(ecg[idx]) for idx in block]
206.             keep = block[int(np.argmax(vals))]
207.             final.append(keep)
208.         i = j
209.     detected = np.array(final, dtype=int)
210.
211.     detected.sort()
212.     if debug:
213.         print(f'[Pan-Tompkins] final detected beats: {len(detected)}')
214.     return detected

```

The above Pan-Tompkins script implements the classic QRS detection algorithm using purely manual digital signal processing (DSP) techniques without relying on `scipy.signal` filter libraries.

1. **Bandpass filtering**, the code manually generates a Finite Impulse Response (FIR) filter kernel using the **windowed Sinc method**. By subtracting a low-pass sinc kernel from a high-pass representation and applying a Hanning window, it isolates the 5–15 Hz frequency band of QRS complex, which contains the maximum energy of the QRS complex while rejecting muscle noise and baseline wander.
2. **Derivative**, a 5-point derivative kernel $[-1, -2, 0, 2, 1]$ is convolved with the signal. This step highlights the slope information, as the R-wave has the steepest slope in the ECG cycle.
3. **Squaring and integration**, the signal is squared ($y[n] = x[n]^2$) to convert all data points to positive values and non-linearly amplify the high-amplitude R-waves. This is followed by a Moving Window Integration (MWI) which sums the energy under the QRS complex, creating a distinct "hump" that is easier to threshold than a sharp single peak.

b. *Segmentation_ECG_to_PCG.py*

```

1. def segment_one_cycle(pcg: np.ndarray,
2.     r_peaks: np.ndarray,
3.     idx: int = 0,
4.     pad_ms: float = 50.0,
5.     fs: int = 2000,
6.     pre_ms: Optional[float] = None,
7.     post_ms: Optional[float] = None,
8.     enforce_single_cycle: bool = True,
9.     allow_partial_edges: bool = True) -> Tuple[np.ndarray, int,
    int]:
10.     if idx < 0 or idx >= len(r_peaks) - 1:
11.         raise IndexError("idx must be within 0..len(r_peaks)-2")

```

```

12.
13. # determine left/right pad in samples
14. if pre_ms is None:
15.     pre_ms = pad_ms
16. if post_ms is None:
17.     post_ms = pad_ms
18. pre_samp = int(round(pre_ms / 1000.0 * fs))
19. post_samp = int(round(post_ms / 1000.0 * fs))
20.
21. # naive start/end (may be trimmed below)
22. start = r_peaks[idx] - pre_samp
23. end = r_peaks[idx + 1] + post_samp
24.
25. # enforce boundaries
26. start = int(max(0, start))
27. end = int(min(len(pcg), end))
28.
29. # If requested, clip to midpoints between adjacent R-peaks to avoid
    crossing into neighbor cycles
30. if enforce_single_cycle:
31.     # compute midpoint to previous R (between r_peaks[idx-1] and
        r_peaks[idx])
32.     if idx > 0:
33.         prev_mid = int(round(0.5 * (r_peaks[idx - 1] + r_peaks[idx])))
34.     else:
35.         prev_mid = 0
36.     # compute midpoint to next next R (between r_peaks[idx+1] and
        r_peaks[idx+2])
37.     if idx + 2 < len(r_peaks):
38.         next_mid = int(round(0.5 * (r_peaks[idx + 1] + r_peaks[idx + 2])))
39.     else:
40.         next_mid = len(pcg)
41.
42.     # If user asked to disallow using partial edges and a midpoint is
        missing near the requested interval,
43.     # raise an error for clarity; otherwise clamp to 0/len(pcg).
44.     if not allow_partial_edges:
45.         if idx == 0 and prev_mid == 0 and (r_peaks[idx] - pre_samp) < 0:
46.             raise IndexError("Left-side midpoint unavailable and
                allow_partial_edges=False")
47.         if (idx + 2) >= len(r_peaks) and next_mid == len(pcg) and
            (r_peaks[idx + 1] + post_samp) > len(pcg):
48.             raise IndexError("Right-side midpoint unavailable and
                allow_partial_edges=False")
49.

```

```

50.     # clip start/end into exclusive midpoints region so we don't include
    neighbor cycles
51.     start = max(start, prev_mid)
52.     end = min(end, next_mid)
53.
54.     # ensure not inverted (rare); if inverted, fallback to original padding
    clipped to signal
55.     if start >= end:
56.         # fallback to minimal safe RR window without padding
57.         start = r_peaks[idx]
58.         end = r_peaks[idx + 1]
59.         # still enforce in-bounds
60.         start = max(0, start); end = min(len(pcg), end)
61.
62.     # final clipping to array bounds (safety)
63.     start = int(max(0, min(start, len(pcg))))
64.     end = int(max(0, min(end, len(pcg))))
65.
66.     return pcg[start:end].copy(), start, end
67.
68.
69. def choose_clean_beat(pcg: np.ndarray,
70.                       r_peaks: np.ndarray,
71.                       fs: int = 2000,
72.                       method: str = 'envelope_ratio',
73.                       envelope_win_ms: float = 50.0,
74.                       search_pad_ms: float = 30.0) -> int:
75.     if r_peaks is None or len(r_peaks) < 2:
76.         return 0
77.     pcg = np.asarray(pcg).ravel()
78.     if pcg.size == 0:
79.         return 0
80.
81.     # prepare envelope (smoothed absolute)
82.     env_win = max(1, int(round(envelope_win_ms / 1000.0 * fs)))
83.     env_kernel = np.ones(env_win) / float(env_win)
84.     env = np.convolve(np.abs(pcg), env_kernel, mode='same')
85.
86.     # pad in samples for local measurement
87.     pad_s = int(round(search_pad_ms / 1000.0 * fs))
88.
89.     scores = []
90.     for i in range(len(r_peaks) - 1):
91.         # obtain candidate segment bounds, but do not apply midpoint clipping
        here
92.         # only want a local window around the RR interval for scoring

```

```

93.     start = max(0, r_peaks[i] - pad_s)
94.     end = min(len(pcg), r_peaks[i + 1] + pad_s)
95.     if end <= start:
96.         scores.append(0.0)
97.         continue
98.     seg_env = env[start:end]
99.     if seg_env.size == 0:
100.        scores.append(0.0)
101.        continue
102.        # compute simple ratio: peak / median (robust)
103.        peak = float(np.max(seg_env))
104.        med = float(np.median(seg_env)) + 1e-12
105.        score = peak / med
106.        scores.append(score)
107.
108.    scores = np.array(scores, dtype=float)
109.    if np.all(np.isfinite(scores)) and scores.size > 0 and
        np.nanmax(scores) > 0:
110.        best_idx = int(np.nanargmax(scores))
111.        return best_idx
112.    else:
113.        return 0

```

This Segmentation script handles the synchronization between the electrical (ECG) and mechanical (PCG) domains of heart signal.

1. **Midpoint clamping**, the most critical logic in `segment_one_cycle` is the enforcement of boundaries based on R-R midpoints. If a fixed time window (for example, 500ms) were used arbitrarily, the S1 sound of the *next* heartbeat might appear at the end of the *current* frame. By calculating the midpoint $\frac{R_i + R_{i+1}}{2}$, the code ensures the segmentation cuts exactly in the diastolic silence between beats.
2. **Clean beat selection**, to automate analysis, `choose_clean_beat` calculates an Envelope SNR. It computes the ratio of the maximum amplitude (Signal) to the median amplitude (Noise floor) for each beat. The cycle with the highest ratio is selected for visualization, ensuring the clearest S1/S2 patterns are presented.

c. *STFT_PCG.py*

```

1. def _get_manual_window(window_name, N):
2.     # Create index array n = 0..N-1
3.     n = np.arange(N, dtype=float)
4.
5.     if window_name is None:
6.         return np.ones(N, dtype=float)
7.
8.     if isinstance(window_name, str):
9.         w_name = window_name.lower()
10.
11.        if 'hann' in w_name:
12.            # Formula: 0.5 - 0.5 * cos(2*pi*n / (N-1))

```

```

13.         if N == 1: return np.ones(1)
14.         return 0.5 - 0.5 * np.cos(2.0 * np.pi * n / (N - 1))
15.
16.     elif 'hamming' in w_name:
17.         # Formula:  $0.54 - 0.46 * \cos(2\pi n / (N-1))$ 
18.         if N == 1: return np.ones(1)
19.         return 0.54 - 0.46 * np.cos(2.0 * np.pi * n / (N - 1))
20.
21.     elif 'blackman' in w_name:
22.         # Formula:  $0.42 - 0.5 * \cos(2\pi n / (N-1)) + 0.08 * \cos(4\pi n / (N-1))$ 
23.         if N == 1: return np.ones(1)
24.         term1 = 0.42
25.         term2 = 0.5 * np.cos(2.0 * np.pi * n / (N - 1))
26.         term3 = 0.08 * np.cos(4.0 * np.pi * n / (N - 1))
27.         return term1 - term2 + term3
28.
29.     elif w_name in ('rect', 'box', 'ones', 'rectangular'):
30.         return np.ones(N, dtype=float)
31.     else:
32.         # Fallback to hann if unknown
33.         if N == 1: return np.ones(1)
34.         return 0.5 - 0.5 * np.cos(2.0 * np.pi * n / (N - 1))
35.
36.     # If user passed a custom array
37.     arr = np.asarray(window_name, dtype=float)
38.     if arr.size != N:
39.         raise ValueError(f"Window length {arr.size} does not match nperseg
40. {N}")
41.     return arr
42.
43. def compute_stft(x, fs=2000, nperseg=None, noverlap=None, nfft=None,
44.                 window='hann', scaling='density', mode='magnitude'):
45.     x = np.asarray(x, dtype=float).ravel()
46.     N = x.size
47.     if N == 0:
48.         return np.zeros((0, 0)), np.zeros((0,)), np.zeros((0,)), "STFT (Manual)"
49.
50.     # Parameter Validation
51.     if nperseg is None:
52.         nperseg = min(256, max(64, max(1, N // 8)))
53.
54.     if nfft is None:
55.         nfft = max(256, nperseg)
56.
57.     if noverlap is None:

```

```

58.     noverlap = int(nperseg // 2)
59.     if noverlap >= nperseg:
60.         noverlap = nperseg // 2
61.
62.     nperseg = int(max(1, nperseg))
63.     nfft = int(max(1, nfft))
64.     noverlap = int(max(0, noverlap))
65.     hop = nperseg - noverlap
66.     if hop <= 0:
67.         hop = 1
68.
69.     # Generate Window
70.     # Manual window creation
71.     try:
72.         win = _get_manual_window(window, nperseg)
73.     except Exception as e:
74.         raise ValueError(f'Invalid window: {e}')
75.
76.     # Padding Logic
77.     if N <= nperseg:
78.         n_frames = 1
79.     else:
80.         n_frames = int(np.ceil((N - noverlap) / float(hop)))
81.
82.     pad_len = (n_frames - 1) * hop + nperseg
83.     if pad_len > N:
84.         x_padded = np.concatenate([x, np.zeros(pad_len - N, dtype=float)])
85.     else:
86.         x_padded = x[:pad_len]
87.
88.     # Main STFT Loop
89.     frames = []
90.     times = []
91.
92.     # Iterate through signal
93.     for i in range(0, pad_len - nperseg + 1, hop):
94.         # 1. Apply Window
95.         seg = x_padded[i : i + nperseg] * win
96.
97.         # 2. Compute Manual FFT
98.         # Returns complex numbers
99.         X_complex = _manual_rfft(seg, nfft)
100.
101.         # 3. Compute Magnitude
102.         # Manual abs: sqrt(real^2 + imag^2)
103.         mag = np.sqrt(X_complex.real**2 + X_complex.imag**2)

```

```

104.
105.     if mode == 'power':
106.         mag = mag ** 2
107.
108.         frames.append(mag)
109.
110.         # Center time calculation
111.         center = (i + (nperseg / 2.0)) / float(fs)
112.         times.append(center)
113.
114.     # Formatting Output
115.     if len(frames) == 0:
116.         freqs = _manual_rfftfreq(nfft, d=1.0/fs)
117.         return np.zeros((freqs.size, 0)), freqs, np.array([], dtype=float),
            "STFT (Manual)"
118.
119.     # Stack frames (Frequency x Time)
120.     S = np.column_stack(frames)
121.
122.     # Generate Frequency Axis
123.     freqs = _manual_rfftfreq(nfft, d=1.0/fs)
124.
125.     # Scaling
126.     # Manual scaling implementation
127.     if scaling == 'density':
128.         # sum of squared window samples
129.         w_energy = np.sum(win ** 2)
130.         if w_energy <= 0:
131.             w_energy = 1.0
132.         S = S / np.sqrt(w_energy)
133.     elif scaling == 'spectrum':
134.         pass
135.
136.     return S, freqs, np.asarray(times, dtype=float), "STFT (Manual)"

```

The STFT_PCG.py script compute a more direct mathematical implementation.

1. **Window generation.** Instead of calling a library, the code generates the Hanning window using the cosine formula $w[n] = 0.5(1 - \cos(\frac{2\pi n}{N-1}))$
2. **Sliding DFT.** The core algorithm iterates through the signal with a step size defined by the hop length. For every step, it extracts a segment, multiplies it by the window, and performs the Fourier Transform.
3. **Manual transformation.** The script uses a recursive Cooley-Tukey algorithm (or direct DFT matrix multiplication for non-power of 2 sizes) to compute the summation of $X[k] = \sum x[n]e^{-j2\pi kn/N}$. This converts the time-domain vibration of the heart sound into frequency components for that specific time slice.

d. *CWT_PCG.py*

```
1. def _get_morlet_kernel(M, w0=5.0):
2.     # Create time vector centered at 0
3.     t = np.arange(-M // 2, M // 2)
4.     # Standardize time for formulation (assuming standard deviation scaling
    happens outside or via window length)
5.     sigma = M / 8.0
6.     t_scaled = t / sigma
7.
8.     # Constant factor  $\pi^{(-1/4)}$ 
9.     c = np.pi ** (-0.25)
10.
11.    # Gaussian envelope:  $\exp(-t^2 / 2)$ 
12.    envelope = np.exp(-0.5 * t_scaled**2)
13.
14.    # Oscillatory part:  $\exp(j * w_0 * t)$ 
15.    # Using Euler's formula:  $\exp(jx) = \cos(x) + j\sin(x)$ 
16.    oscillation = np.cos(w0 * t_scaled) + 1j * np.sin(w0 * t_scaled)
17.
18.    psi = c * envelope * oscillation
19.    return psi
20. def _compute_cwt_manual(signal, fs, fmin, fmax, n_freqs, w0=5.0):
21.     signal = np.asarray(signal)
22.     N = signal.size
23.     dt = 1.0 / fs
24.
25.     # 1. Generate Frequency Vector (Linear Space)
26.     freqs = np.linspace(fmin, fmax, n_freqs)
27.
28.     # 2. Convert to Scales
29.     # Relation for Morlet:  $f = (w_0 * fs) / (2 * \pi * a)$  (approx)
30.     # Thus:  $a = (w_0 * fs) / (2 * \pi * f)$ 
31.     scales = (w0 * fs) / (2.0 * np.pi * freqs)
32.
33.     # Prepare output matrix
34.     # Scalogram: Rows = Frequencies, Cols = Time
35.     cwt_matrix = np.zeros((n_freqs, N), dtype=float)
36.
37.     # 3. and 4. Loop over scales (Direct implementation)
38.     for i, a in enumerate(scales):
39.         # Determine effective support of the wavelet at this scale
40.         # width  $\sim a * \text{base\_width}$ .
41.         window_size = int(min(10 * a, N)) # Limit window to signal size
42.         if window_size % 2 == 0:
43.             window_size += 1 # Ensure odd length for centering
44.
```

```

45.     # In CWT formula:  $\psi_{a,b}(t) = (1/\sqrt{a}) * \psi((t-b)/a)$ 
46.
47.     # Base kernel (time scaling handles the 'a' in denominator)
48.     kernel = _get_morlet_kernel(window_size, w0=w0)
49.
50.     # Energy Normalization ( $1/\sqrt{a}$ )
51.     norm_factor = 1.0 / np.sqrt(a)
52.     kernel = kernel * norm_factor
53.
54.     # Convolve
55.     conv_res = _manual_convolution(signal, kernel)
56.
57.     # 5. Compute Power (Magnitude Squared)
58.     #  $|W(a,b)|^2 = \text{Real}^2 + \text{Imag}^2$ 
59.     power = (conv_res.real ** 2) + (conv_res.imag ** 2)
60.     cwt_matrix[i, :] = power
61.
62.     times = np.arange(N) * dt
63.     return cwt_matrix, freqs, times

```

This above script implements the Continuous Wavelet Transform or CWT from first principles.

1. **Morlet wavelet construction.** The code manually generates the mother wavelet $\psi(t)$ by multiplying a Gaussian envelope ($e^{-t^2/2}$) with a complex sinusoid ($e^{j\omega_0 t}$). This shape is chosen because it closely resembles the impulsive, decaying nature of heart sounds.
2. **Scale-Frequency relationship.** The code explicitly converts the target frequencies into scales using the relationship $a \propto \frac{f_s}{f}$.
3. **Convolution.** The CWT is computed via convolution. For each scale a , the wavelet is stretched (dilated). The convolution of this stretched wavelet with the signal measures the similarity between the signal and the wavelet at that specific scale. The magnitude squared of this result yields the scalogram (power distribution).

e. *Threshold_Plot_CoG.py*

```

1. def _label_components(bin_mask):
2.     mask = np.asarray(bin_mask, dtype=bool)
3.     h, w = mask.shape
4.     labels = np.zeros((h, w), dtype=np.int32)
5.     label = 0
6.     # neighbors offsets (8-connectivity)
7.     neigh = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
8.     for i in range(h):
9.         for j in range(w):
10.             if not mask[i, j] or labels[i, j] != 0:
11.                 continue
12.             label += 1

```

```

13.         # flood fill using stack
14.         stack = [(i, j)]
15.         labels[i, j] = label
16.         while stack:
17.             y, x = stack.pop()
18.             for dy, dx in neigh:
19.                 ny, nx = y + dy, x + dx
20.                 if 0 <= ny < h and 0 <= nx < w and mask[ny, nx] and labels[ny,
    nx] == 0:
21.                     labels[ny, nx] = label
22.                     stack.append((ny, nx))
23.     return labels, label
24.
25. def threshold_mask(scalogram, thr_ratio, min_area=None, keep_top=3,
    freqs=None, times=None, time_window=None):
26.     S = np.asarray(scalogram)
27.     if S.ndim != 2:
28.         raise ValueError("scalogram must be 2D")
29.     n_freqs, n_times = S.shape
30.     maxS = np.nanmax(S)
31.     if maxS == 0 or np.isnan(maxS):
32.         return np.zeros_like(S, dtype=bool)
33.     global_thr = float(thr_ratio) * float(maxS)
34.     base_mask = (S >= global_thr)
35.
36.     # label components
37.     labeled, ncomp = _label_components(base_mask)
38.     if ncomp == 0:
39.         return np.zeros_like(S, dtype=bool)
40.
41.     # compute areas
42.     areas = []
43.     for idx in range(1, ncomp+1):
44.         ys, xs = np.nonzero(labeled == idx)
45.         areas.append(len(ys))
46.     areas = np.array(areas, dtype=int)
47.
48.     if min_area is None:
49.         min_area_adaptive = max(3, int(0.005 * S.size))
50.     else:
51.         min_area_adaptive = int(min_area)
52.
53.     keep_mask = np.zeros_like(S, dtype=bool)
54.     for idx, area in enumerate(areas, start=1):
55.         if area >= min_area_adaptive:
56.             keep_mask |= (labeled == idx)

```

```

57. if not keep_mask.any():
58.     order = np.argsort(-areas)
59.     for k in range(min(keep_top, len(order))):
60.         comp_idx = int(order[k]) + 1
61.         keep_mask |= (labeled == comp_idx)
62.
63. # time window refinement (compute centroid times for components)
64. if time_window is not None and times is not None:
65.     t0, t1 = float(time_window[0]), float(time_window[1])
66.     comps = np.unique(labeled[keep_mask])
67.     refined_mask = np.zeros_like(keep_mask, dtype=bool)
68.     times_arr = np.asarray(times, dtype=float)
69.     for comp in comps:
70.         if comp == 0:
71.             continue
72.         comp_mask = (labeled == comp)
73.         if not comp_mask.any():
74.             continue
75.         ys, xs = np.nonzero(comp_mask)
76.         if xs.size == 0:
77.             continue
78.         col_mean = float(xs.mean())
79.         col_idx = int(round(col_mean))
80.         col_idx = max(0, min(col_idx, len(times_arr)-1))
81.         t_comp = float(times_arr[col_idx])
82.         if (t_comp >= t0) and (t_comp <= t1):
83.             refined_mask |= comp_mask
84.     return refined_mask.astype(bool)
85.
86. return keep_mask.astype(bool)
87.
88. def compute_cog(scalogram, freqs, times, mask=None):
89.     S = np.asarray(scalogram, dtype=float)
90.     if S.ndim != 2:
91.         raise ValueError("scalogram must be 2D")
92.     n_freqs, n_times = S.shape
93.     if freqs is None or len(freqs) != n_freqs:
94.         freqs = np.linspace(0.0, 1.0, n_freqs)
95.     if times is None or len(times) != n_times:
96.         times = np.linspace(0.0, float(n_times-1), n_times)
97.     freqs = np.asarray(freqs, dtype=float)
98.     times = np.asarray(times, dtype=float)
99.
100.     if mask is None:
101.         M = np.ones_like(S, dtype=bool)
102.     else:

```

```

103.         M = _ensure_mask_shape(mask, (n_freqs, n_times))
104.
105.         W = np.abs(S) * M
106.         total = np.nansum(W)
107.         if not np.isfinite(total) or total <= 0:
108.             return None
109.
110.         time_weights = np.nansum(W, axis=0)
111.         freq_weights = np.nansum(W, axis=1)
112.
113.         t_cog = float(np.nansum(time_weights * times) /
114.                        np.nansum(time_weights))
114.         f_cog = float(np.nansum(freq_weights * freqs) /
115.                        np.nansum(freq_weights))
115.
116.         return (t_cog, f_cog)

```

This script will act as the feature extraction engine for the time-frequency analysis. To identify distinct S1 and S2 regions, the script implements a manual **Connected Component Labeling** algorithm. This function uses a stack-based flood-fill approach with 8-connectivity to traverse the binary threshold mask, grouping adjacent pixels into unique "blobs" and filtering out small noise artifacts based on their area. Once the regions are defined, the `compute_cog` function calculates the **Center of Gravity (CoG)** using the geometric moment method. By treating the spectral power at each pixel as "mass," it computes the weighted average of the time and frequency coordinates, providing a precise, mathematically defined centroid for each heart sound.

f. *GUI.py*

```

1.  class PCGAnalyzerGUI(QtWidgets.QMainWindow):
2.      def __init__(self):
3.          # ... (UI Initialization) ...
4.          self._build_ui()
5.          self._connect_signals()
6.
7.      def _on_run_pt(self):
8.          # Calls the manual Pan-Tompkins module
9.          r_peaks = detect_r_peaks_with_fallback(ecg, fs=self.fs)
10.
11.      def _update_segment_and_downstream(self, beat_idx):
12.          # Calls Segmentation, CWT/STFT, and CoG modules based on user input
13.          seg, start, end = segment_one_cycle(pcg, self.r_peaks, idx=beat_idx, ...)
14.          if tf_method == 'CWT':
15.              scalogram, ... = compute_cwt(seg, ...)
16.          else:
17.              res = compute_stft(seg, ...)
18.
19.      def _plot_cwt_3d(self, ...):

```

```
20.      # Renders 3D surface plots using Matplotlib backend for Qt
21.      surf = self.ax_cwt_3d.plot_surface(Ts, Fs, Zs, ...)
```

The GUI.py script is the core of this program's user interface, built using the PyQt6 library. This script acts as a controller that integrates all previously created manual signal processing modules. Its main functions are:

1. Module integration, that connects user interactions (buttons, sliders) with backend functions such as `detect_r_peaks` (Pan-Tompkins), `compute_cwt`, and `threshold_mask`.
2. Data visualization, uses the Matplotlib backend embedded in the Qt widget to render real-time signals, 2D spectrograms, and interactive 3D visualizations of the time-frequency transformation results.
3. State management, manages the data flow from raw file reading, per-beat segmentation, to dynamic Center of Gravity (CoG) calculations as parameters are changed by the user.

g. *main.py*

```
1. #Start the GUI app
2. import sys
3. from PyQt6 import QtWidgets
4. from GUI import PCGAnalyzerGUI
5.
6. def main():
7.     app = QtWidgets.QApplication(sys.argv)
8.     win = PCGAnalyzerGUI()
9.     win.show()
10.    sys.exit(app.exec())
11.
12. if __name__ == "__main__":
13.     main()
```

The main.py script serves as the entry point (main execution point) of the software. This script is very simple because its task is only to initialize the application's event loop using `QApplication` and call the main `PCGAnalyzerGUI` class to display the program window. This separation is done to separate the interface logic (GUI.py) from the program's execution logic, adhering to good programming standards.

2.3. Result of the Program and Analysis

*Initial GUI

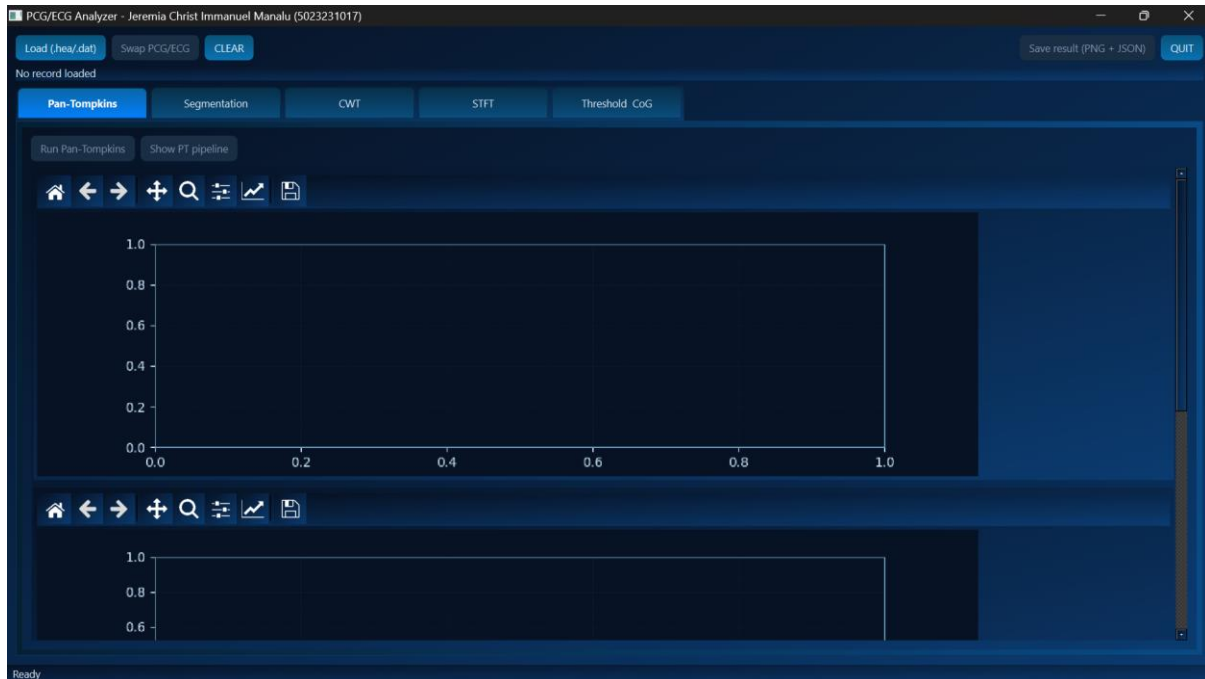


Figure 2.1. Initial GUI Components for the Program

In **Figure 2.1**, it illustrates the initial user interface of the PCG/ECG Analyzer program upon launch. The Graphical User Interface (GUI) is designed with a dark-themed color palette to reduce visual fatigue during prolonged analysis. The interface is structured into a logical workflow using a tabbed navigation system located at the top left, which includes "Pan-Tompkins" for ECG processing, "Segmentation" for beat isolation, "CWT" and "STFT" for time-frequency analysis, and "Threshold CoG" for feature extraction.

The top control bar provides essential file management functions, allowing the user to load physiological data files (.hea or .dat format) and clear the current session. The main display area currently shows empty axes, waiting for signal input. This layout ensures that the user can sequentially process the data from raw signal acquisition to the final center of gravity calculation without needing to switch between different software windows.

2.3.1. Program Results

The data used in this section is a0002.dat (can also use a0002.hea). The detail of the .hea file for a0002 is as below:

a0002 2 2000 41657

a0002.wav 16+44 1 16 0 0 0 0 PCG

a0002.dat 16 1000 16 0 1 24070 0 ECG

Abnormal

1. Pan-Tompkins Tab

The data used in this section is a0002.dat (associated with a0002.he). This record represents an **abnormal** heart condition. The dataset details include a sampling frequency of 2000 Hz, with PCG recorded on channel 0 and ECG on channel 1.

The Pan-Tompkins algorithm is a main method of this analysis, serving as the synchronization trigger for the entire pipeline. Its primary goal is to accurately detect the QRS complexes in the ECG signal, specifically the R-peaks, which mark the onset of ventricular depolarization. As shown in **Figure 2.2**, the raw ECG signal contains baseline wander and high-frequency noise that can interfere with detection. To address this, the algorithm first applies a bandpass filter (5–15 Hz) to isolate the energy of the QRS complex, followed by a derivative operation to highlight the steep slopes characteristic of the R-wave.

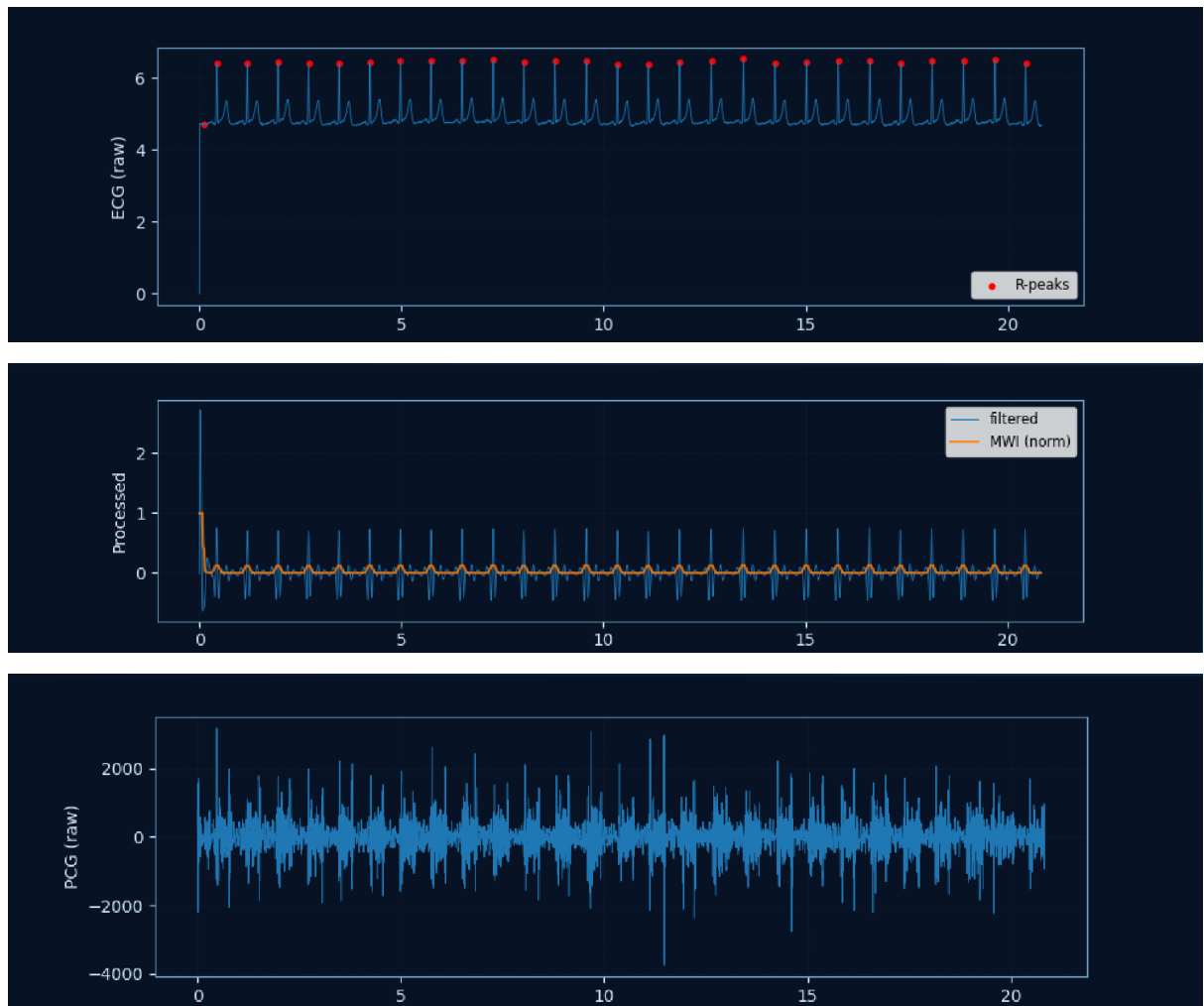


Figure 2.2. Three Graphs Result of the Pan-Tompkins Tab

Figure 2.2 displays the intermediate steps of this process. The top graph shows the filtered signal, which is cleaner than the raw input. **Figure 2.3** visualizes the subsequent steps, the signal is squared to non-linearly amplify the higher amplitude R-waves and convert all data to positive values. The bottom graph of Figure 2.4 displays the result of the Moving Window Integration (MWI). This step aggregates the energy of the squared signal, creating a smooth, bell-shaped wave that makes thresholding robust against false peaks. The final result, seen in the top graph of Figure 2.3s, shows red dots accurately marking the R-peaks,

demonstrating the algorithm's ability to reliably identify cardiac cycles even in abnormal data.

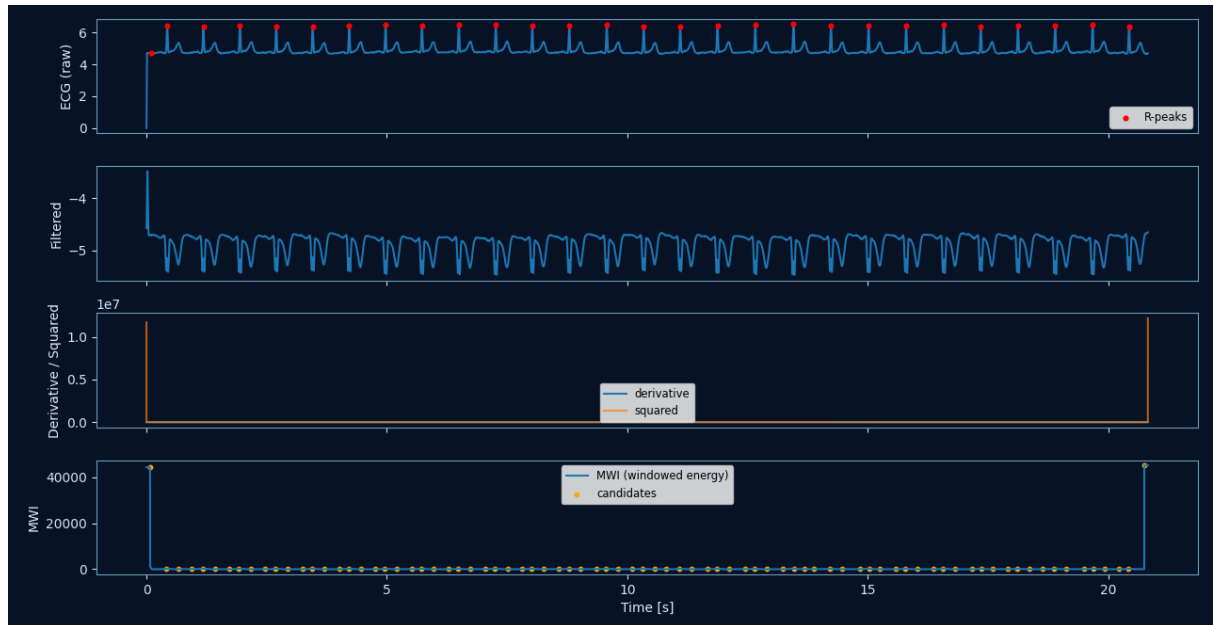
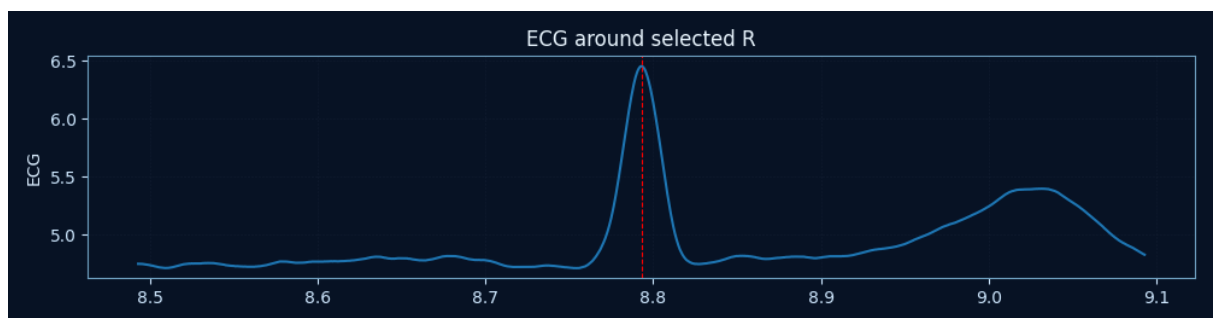


Figure 2.3. *The Result of the Full Pan-Tompkins Algorithm*

2. Segmentation Tab

Following R-peak detection, the system proceeds to segmentation, as shown in **Figure 2.4**. This step is critical for isolating the mechanical heart sounds (S1 and S2) corresponding to a specific electrical activation. The figure focuses on Beat 12. The top graph displays the ECG signal zoomed in around the selected R-peak ($t \approx 8.8s$). The vertical red line indicates the exact time index of the R-peak ($t = 0$ for the segment).

The bottom graph displays the segmented PCG signal. The algorithm determines the cut-off points by calculating the midpoints between the current R-peak and its neighbors. This ensures that the window captures the full systolic and diastolic duration of the current beat without including the S1 sound of the subsequent beat. In the PCG waveform, we can observe the high-frequency oscillations. The first burst of energy shortly after the R-peak corresponds to S1 (AV valve closure), while the second burst later in the cycle corresponds to S2 (semilunar valve closure).



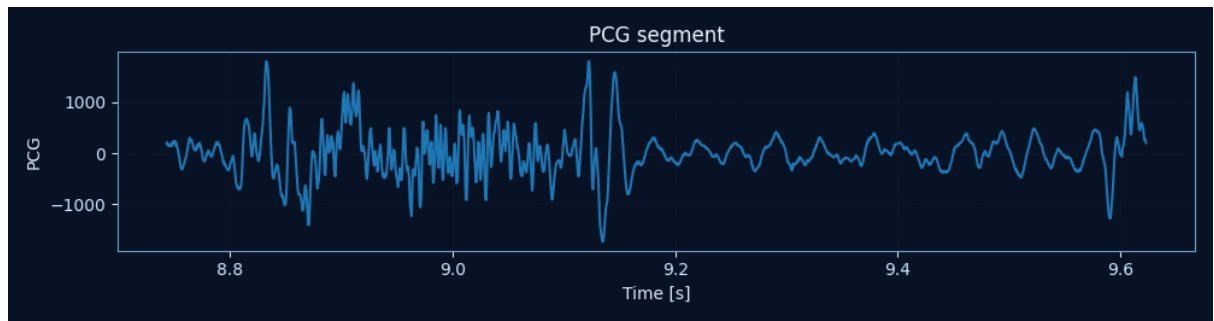
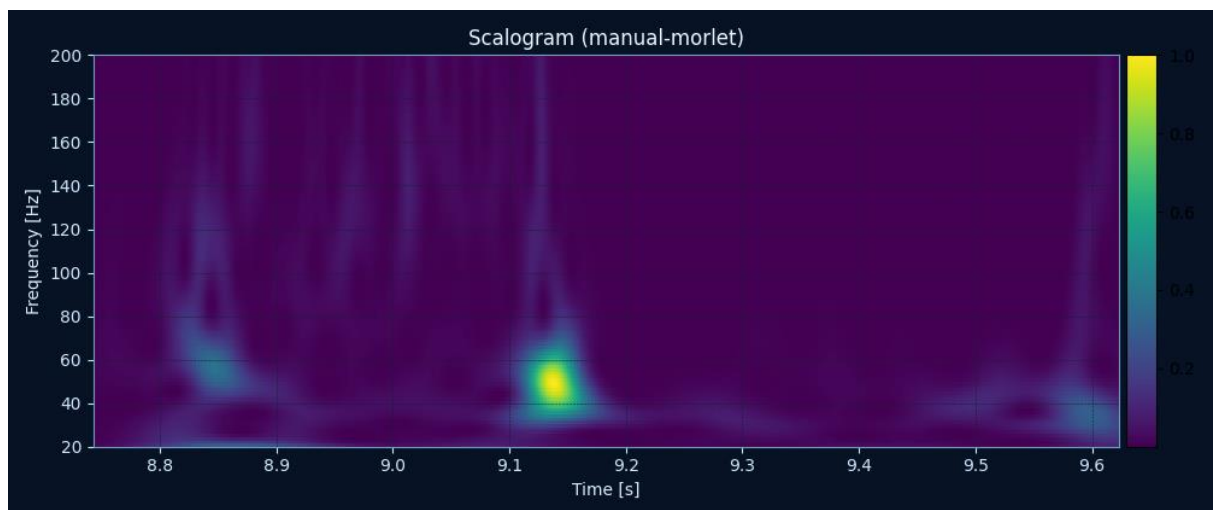


Figure 2.4. The Segmentation Result for (ECG) *Beat 12*

3. CWT Tab

Figure 2.5 presents the results of the Continuous Wavelet Transform (CWT) using the Morlet wavelet. The upper panel shows the 2D Scalogram (Time vs. Frequency), while the lower panel shows the 3D visualization. In the 2D contour plot, the color intensity represents the magnitude of the signal's energy. We can clearly distinguish two vertical "blobs" of high energy (bright yellow/green). The first blob, occurring around $t = 9.15s$, represents the S1 sound, and the second, smaller blob represents the S2 sound.

The Morlet wavelet is particularly effective here because its shape closely resembles the transient, decaying nature of heart sounds, providing excellent time resolution. The 3D surface plot enhances this analysis by adding "height" to the energy representation. It allows the user to visually compare the relative power of S1 versus S2. In this abnormal record (a0002), the energy distribution appears relatively concentrated, though the texture of the scalogram suggests some underlying noise or murmur components typical of the abnormality.



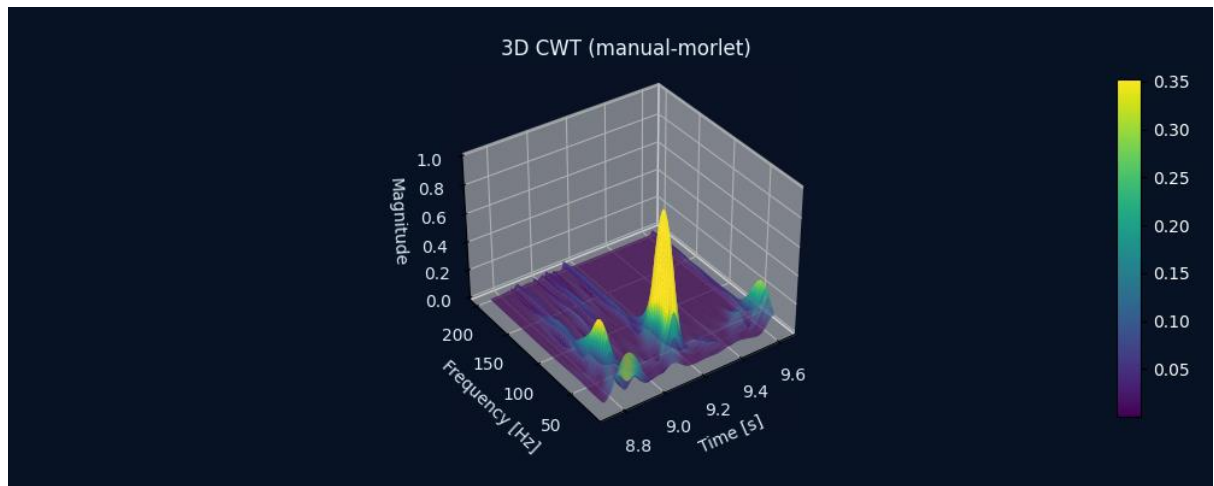
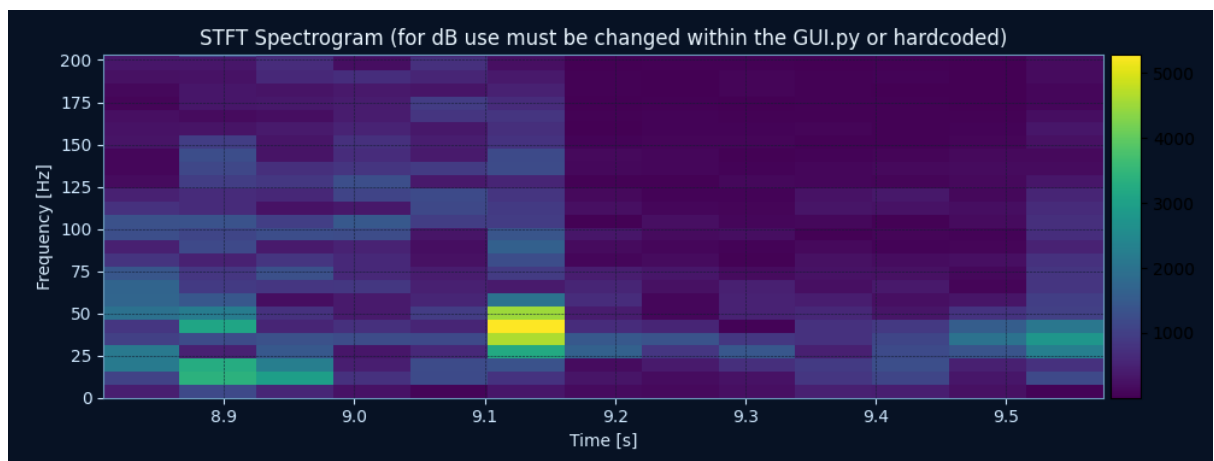


Figure 2.5. The CWT Result for both 2D-Contour and 3D of Beat 12

4. STFT Tab

Figure 2.6 displays the analysis of the same beat, that is Beat 12 using the Short-Time Fourier Transform (STFT). Comparing this to the CWT result reveals the fundamental trade-off in time-frequency analysis. The STFT spectrogram appears "blocky" or pixelated. This is due to the fixed window size used in the FFT calculation; a fixed window imposes a limit on simultaneous time and frequency resolution (Heisenberg uncertainty).

While the STFT successfully identifies the general location of S1 and S2 (indicated by the yellow blocks), the boundaries are less defined than in the CWT scalogram. The "smearing" of energy in the time domain is more apparent. The 3D STFT plot confirms this, showing sharp, rectangular peaks rather than the smooth, organic hills seen in the CWT 3D plot. However, STFT remains computationally faster and provides a direct representation of frequency content, serving as a useful comparison validation for the CWT results.



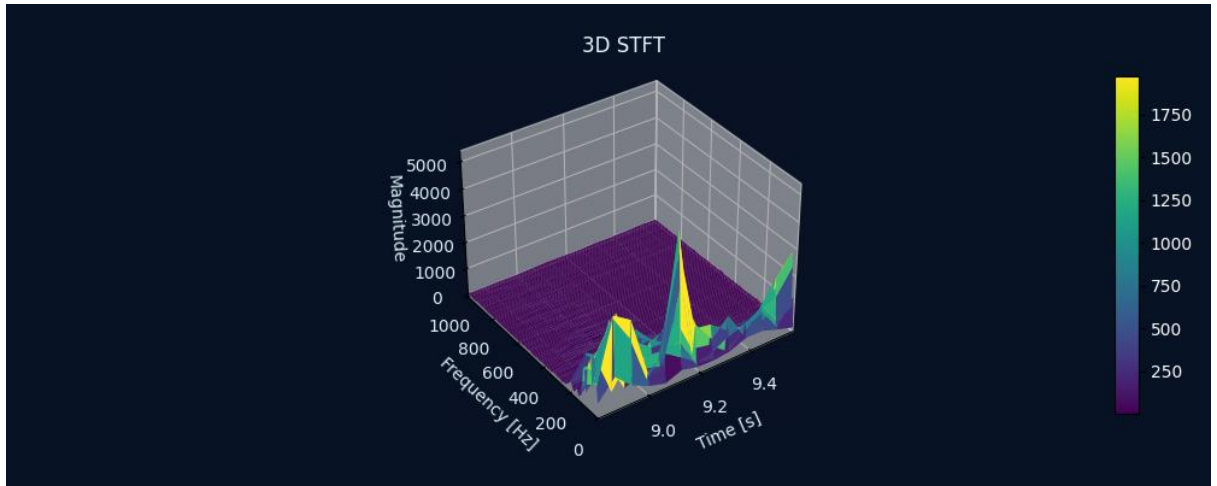
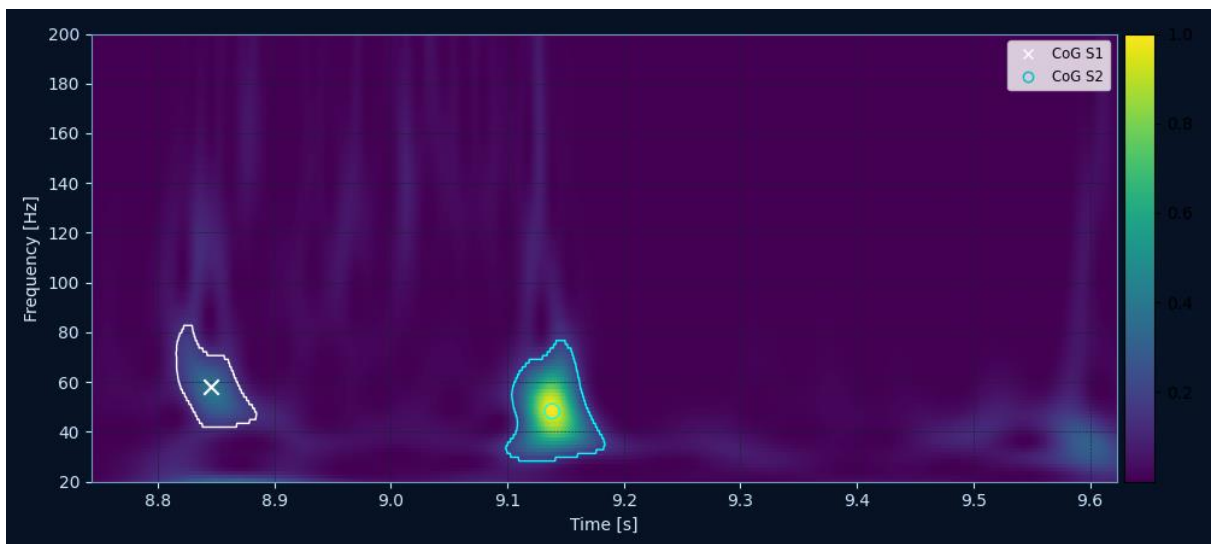


Figure 2.6. The STFT Result for both 2D-Contour and 3D of Beat 12

5. Threshold and CoG Tab

Figure 2.7 illustrates the final feature extraction step, Thresholding and Center of Gravity (CoG) calculation. The top panel shows the CWT scalogram, and the bottom panel shows the STFT spectrogram. The algorithm applies a threshold (60% of peak energy) to create a binary mask, outlined by the white (for S1) and cyan (for S2) contours.

This figure clearly highlights the superior precision of CWT for this application. The contours in the CWT plot are smooth and tightly hug the heart sound energy, whereas the STFT contours are jagged and rectangular. The markers ("X" for S1 and "O" for S2) represent the calculated CoG. This calculation physically represents the weighted average position of the heart sound in time and frequency. By acquiring these coordinates, the program successfully converts the visual signal into quantitative data points (t_{cog} , f_{cog}) that can be used for further medical diagnosis.



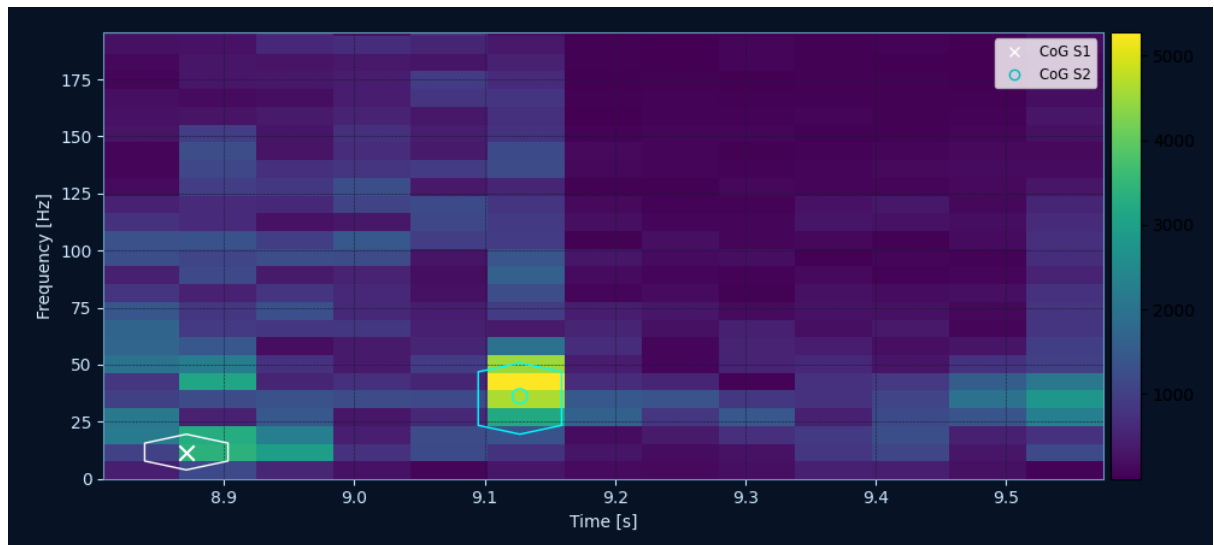


Figure 2.7. The Comparison between CWT and STFT use in CoG Tab

2.3.2. Comparison between Normal and Abnormal PCG Data

The data used in this section is a0002.dat (can also use a0002.heg). The detail of the .heg file for a0012 for normal condition and a0006 for abnormal condition. The details for both .heg data are as below:

a0006 2 2000 41518

a0006.wav 16+44 1 16 0 0 0 0 PCG

a0006.dat 16 1000 16 0 1 9573 0 ECG

#Abnormal

a0012 2 2000 61719

a0012.wav 16+44 1 16 0 0 0 0 PCG

a0012.dat 16 1000 16 0 2 -28761 0 ECG

#Normal

1. Normal Condition

Figure 2.8 shows the global view of the Normal data (a0012). The ECG R-peaks (indicated with red dots) are highly regular, indicating a stable heart rhythm. The PCG amplitude is consistent across beats. In the segmentation of Beat 15 in **Figure 2.9**, the PCG signal shows very distinct, sharp transients for S1 and S2 with a near-silent period (isoelectric line) in between. This silence indicates the absence of murmurs (turbulent blood flow) between valve closures.

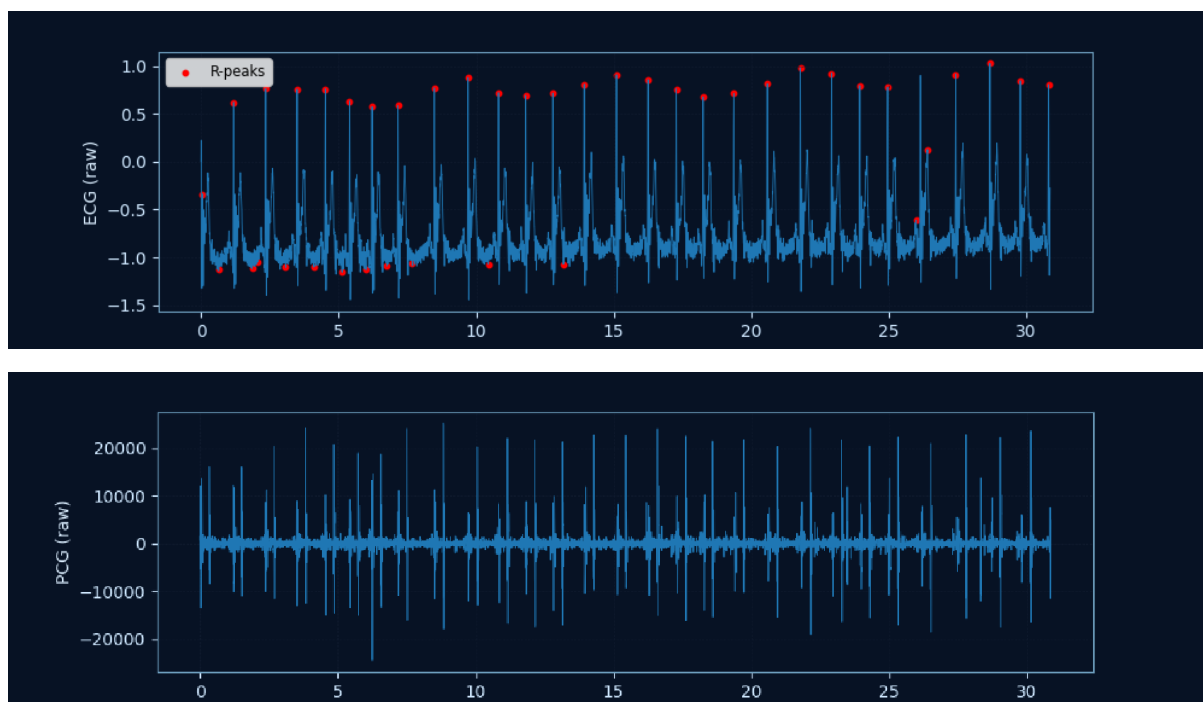


Figure 2.8. *The ECG and PCG Data*

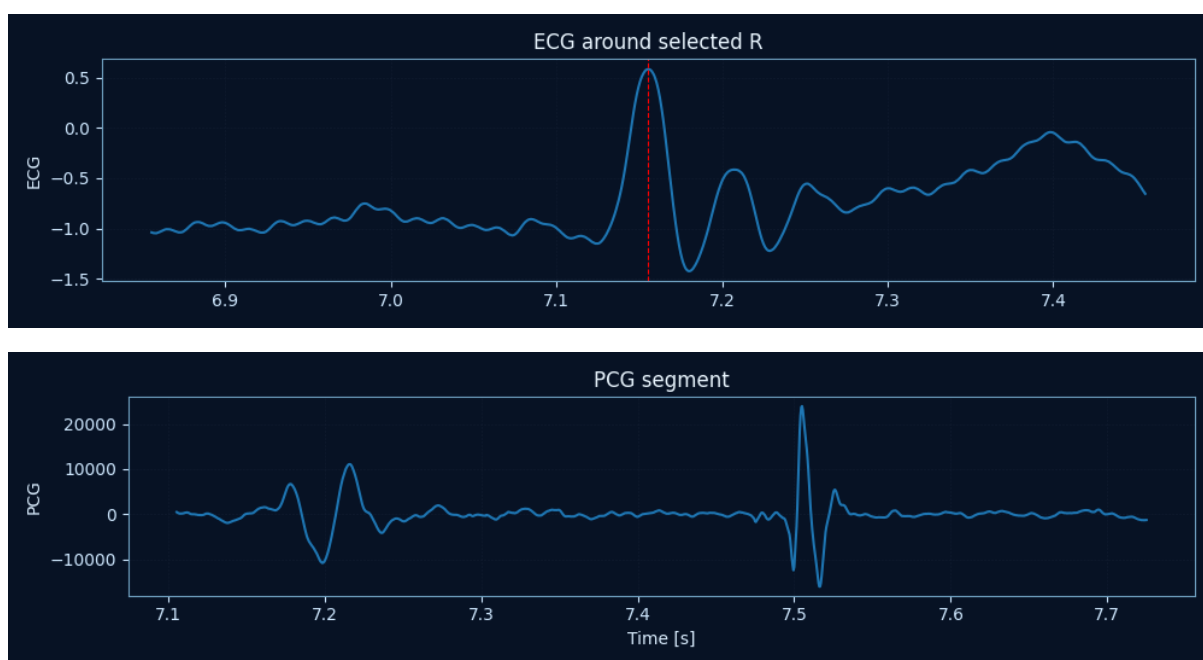


Figure 2.9. *The Segmentation Result for (ECG) Beat 15*

The Time-Frequency analysis in **Figure 2.10** confirms this normality. Both the CWT (top) and STFT (bottom) show two well-separated islands of energy. There is very little "spectral leakage" or noise bridging the gap between S1 and S2. Finally, **Figure 2.11** shows the CoG results. The detected regions (bounded by the white/cyan lines) are compact and distinct. The CoG markers are centrally located within the energy blobs, indicating that the heart sounds are crisp and well-defined, which is the hallmark of a healthy heart.

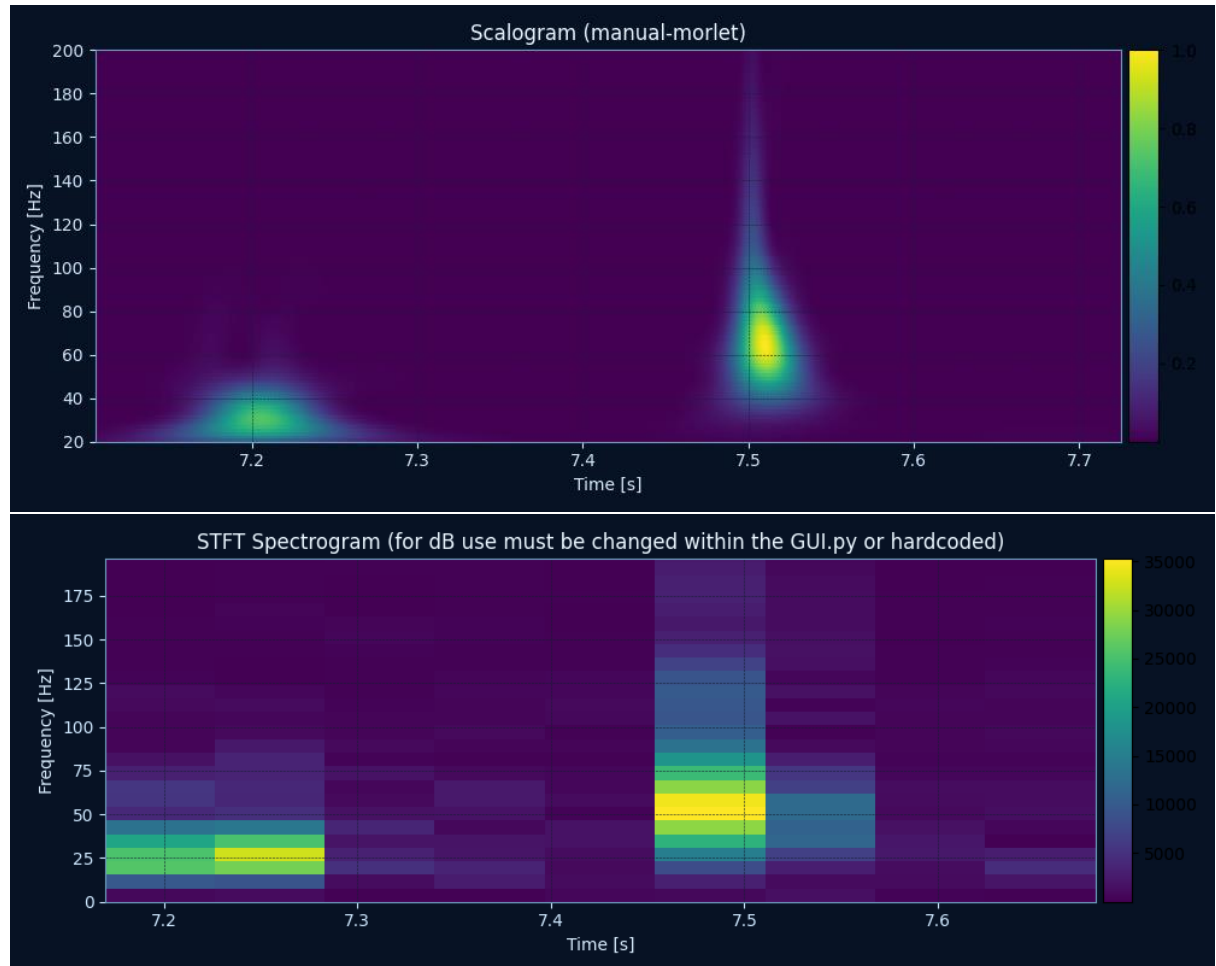


Figure 2.10. The CWT and STFT Result for Beat 15

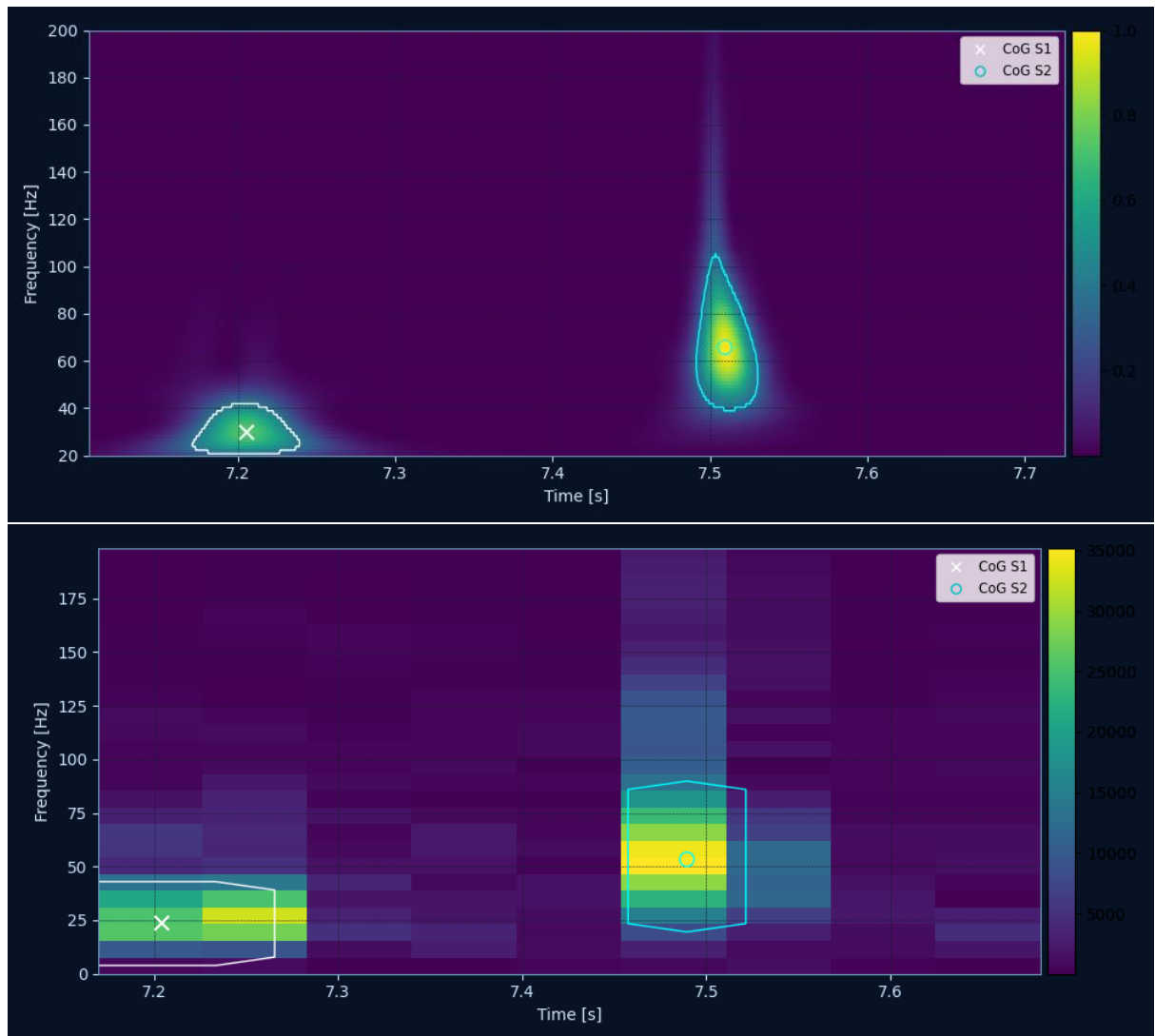
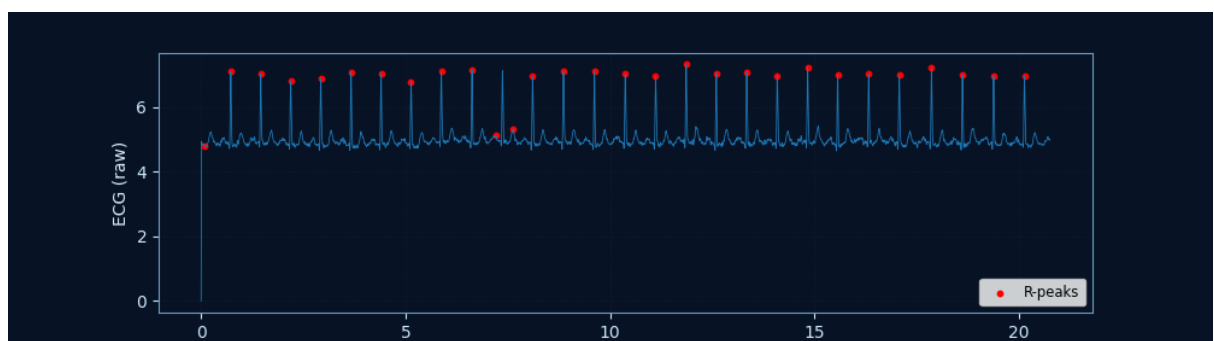


Figure 2.11. The CoG Result for Both CWT and STFT Result for (ECG) Beat 15

2. Abnormal Condition

In contrast of the Normal data, the Abnormal data (a0006) presents a much more chaotic picture. **Figure 2.12** shows the raw signals, the PCG waveform lacks the clear "lub-dub... silence" pattern of the normal record, appearing instead as a continuous noisy signal. The segmentation of Beat 14 (**Figure 2.13**) reveals high-amplitude vibrations persisting throughout the cycle, making visual identification of S1 and S2 difficult in the time domain alone.



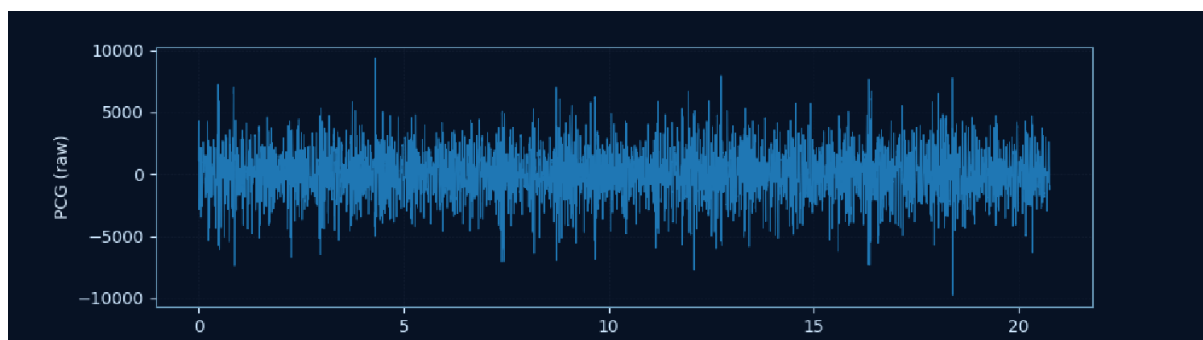


Figure 2.12. The ECG and PCG Data

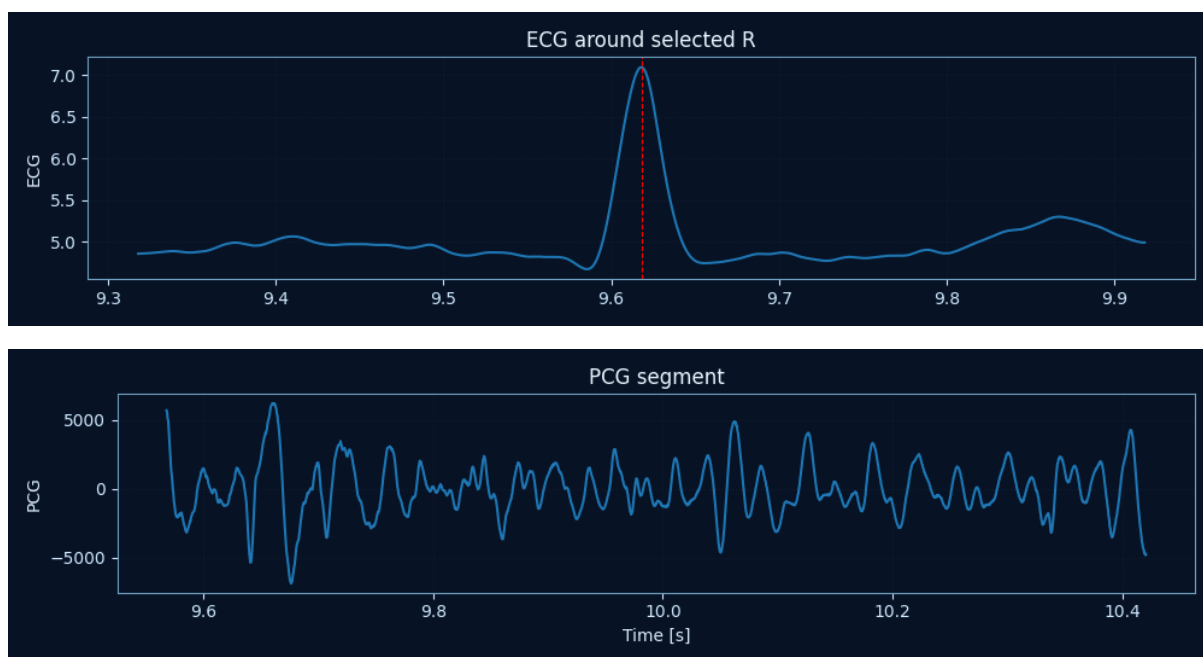


Figure 2.13. The Segmentation Result for (ECG) Beat 14

The CWT and STFT results in **Figure 2.14** dramatically illustrate the pathology. Instead of two distinct islands, we see a "smearing" of energy. The regions corresponding to S1 and S2 are elongated, and there is significant energy present in the frequency bands during the intervals that should be silent. This pattern is characteristic of heart murmurs (e.g., mitral regurgitation or aortic stenosis). Consequently, the Threshold and CoG result in **Figure 2.15** shows irregular, expanded contours. The algorithm detects larger areas of activity, and the Center of Gravity may be shifted by the energy of the murmur. This comparison demonstrates the program's effectiveness: while it precisely localizes sounds in normal hearts, it successfully visualizes and quantifies the spectral disorder present in abnormal heart conditions.

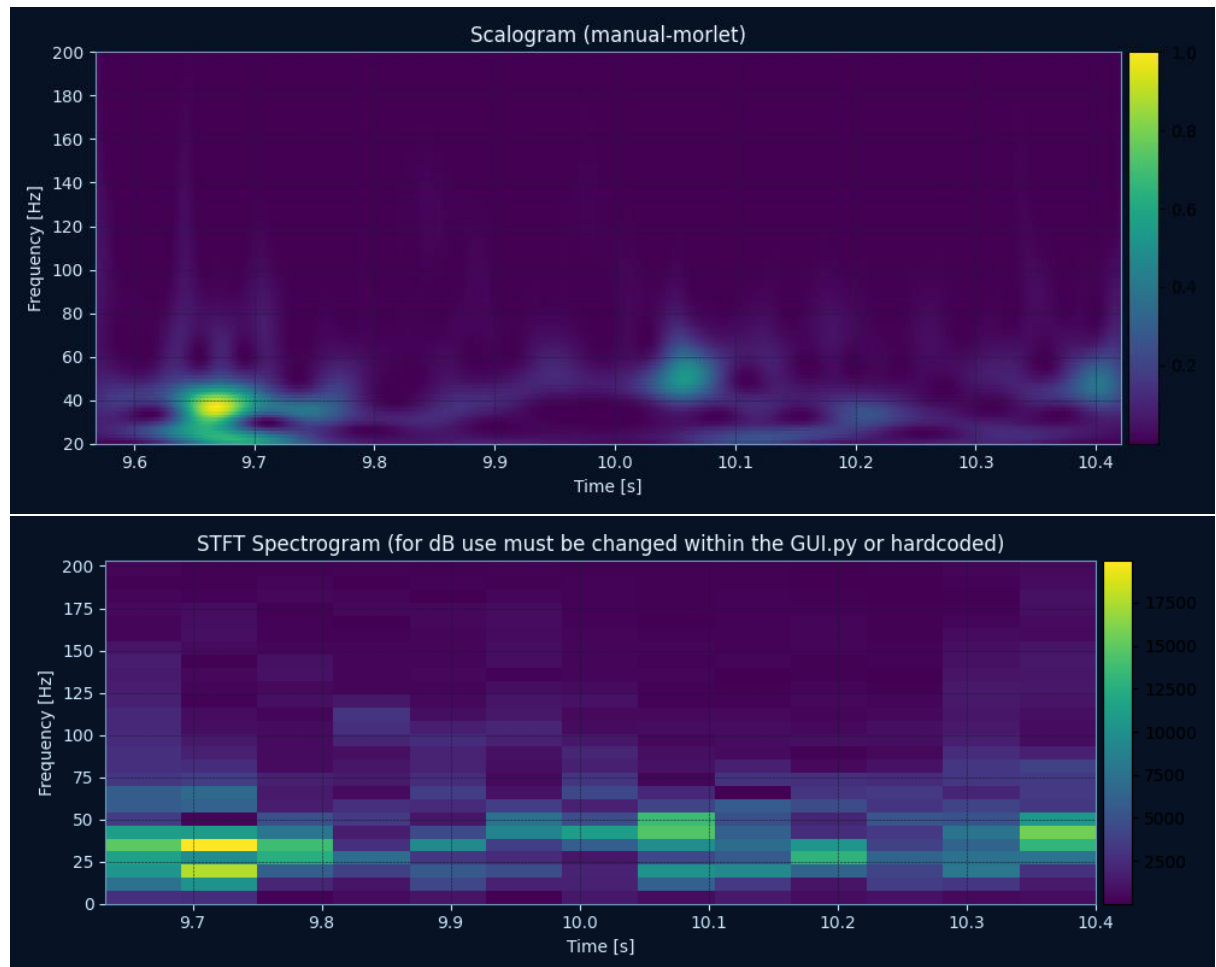


Figure 2.14. The CWT and STFT Result for Beat 14

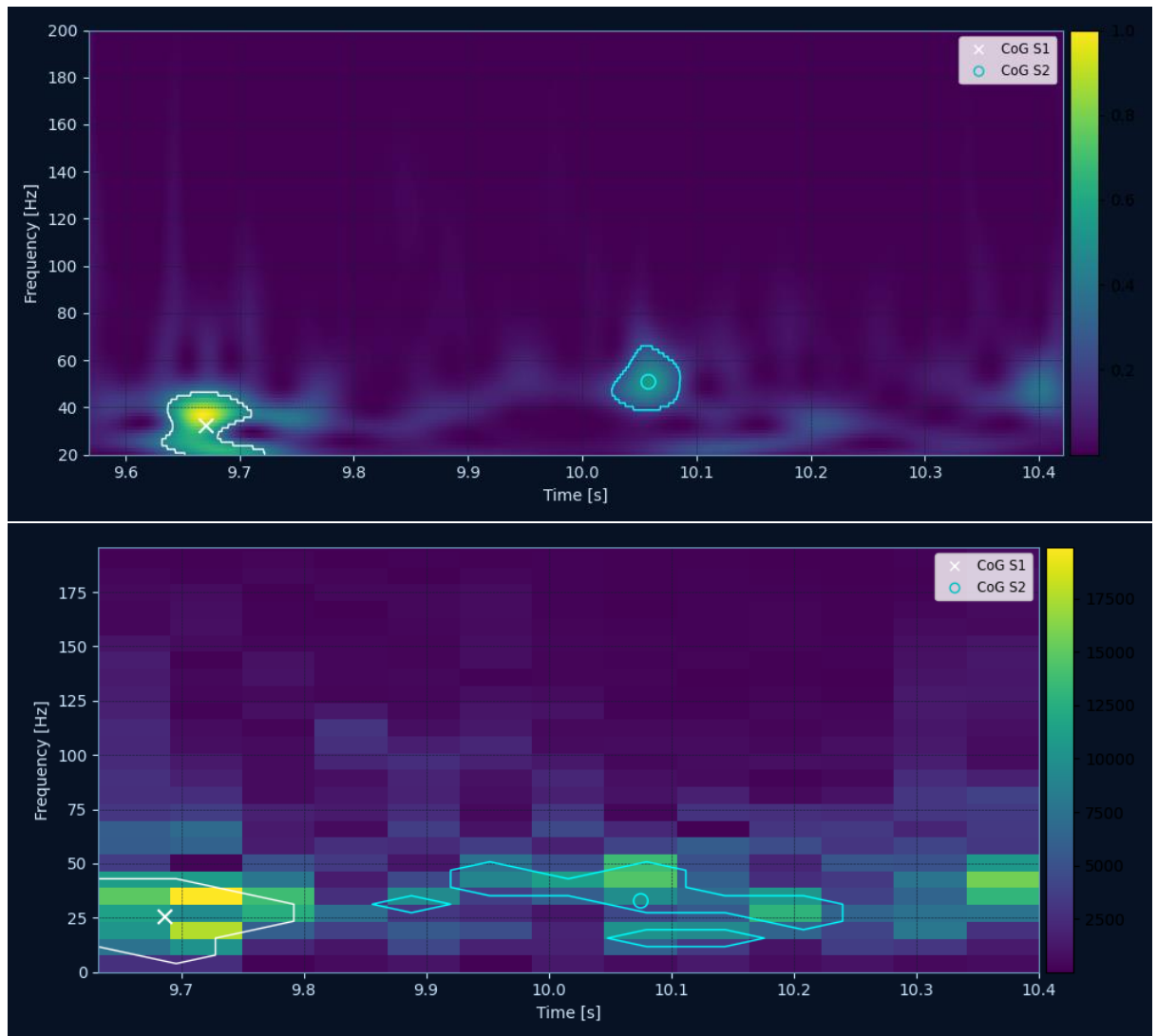


Figure 2.15. The CoG Result for Both CWT and STFT Result for (ECG) Beat 14

2.3.9. Evaluation

The program successfully integrates a comprehensive pipeline to analyze ECG and PCG signals. By synchronizing mechanical heart sounds with electrical R-peak triggers, the system automates the isolation of individual cardiac cycles with high precision. The dual approach of employing Short-Time Fourier Transform (STFT) and Continuous Wavelet Transform (CWT) provides a robust comparative analysis, where the subsequent application of thresholding and Center of Gravity (CoG) algorithms proves effective in quantifying the exact time-frequency location of S1 and S2 sounds for both normal and abnormal pathologies. The key points of evaluation are as follow:

- Robust preprocessing. The manual implementation of the Pan-Tompkins algorithm reliably detects R-peaks even in signals with baseline wander, ensuring accurate time-synchronization for PCG segmentation.
- CWT superiority. The manual Morlet Wavelet implementation demonstrates superior time-frequency resolution compared to STFT, providing smoother and more precise contours for heart sound localization.

- Quantitative feature extraction. The geometric Center of Gravity (CoG) method successfully converts visual spectral energy into quantifiable coordinates (t_{cog}, f_{cog}), enabling the distinct separation of S1 and S2 energy blobs.
- Diagnostic capability. The system effectively distinguishes between healthy and pathological records, clearly visualizing heart murmurs as "spectral smearing" in the time-frequency domain, validating its utility as a diagnostic support tool.

CHAPTER III. CONCLUSION

The program successfully demonstrated and showed the development and implementation of the joint analysis of Electrocardiogram (ECG) and Phonocardiogram (PCG) signals. The project validated the fundamental formulas in signal processing, specifically the Pan-Tompkins algorithm for R-peak detection, the Short-Time Fourier Transform (STFT), and the Continuous Wavelet Transform (CWT). The results indicate that using the ECG R-peak as a time synchronization trigger is a highly effective method for segmenting the non-stationary PCG signal into individual cardiac cycles. Furthermore, the comparative analysis between Time-Frequency methods revealed that the Continuous Wavelet Transform (using the Morlet wavelet) provides superior resolution for resolving the transient nature of S1 and S2 heart sounds compared to the fixed resolution STFT. The integration of the Center of Gravity (CoG) calculation proved to be a robust feature extraction technique, offering precise quantitative coordinates for heart sounds and successfully differentiating between the compact spectral signatures of normal heartbeats and the chaotic energy distributions typical of abnormal heart murmurs or regurgitations.

REFERENCES

- [1] Liu, C., Springer, D., Li, Q., Moody, B., Juan, R.A., Chorro, F.J., Castells, F., Roig, J.M., Silva, I., Johnson, A.E. and Mark, R.G., 2016. An open access database for the evaluation of heart sound algorithms. *Physiological Measurement*, 37(12), p.2181.
- [2] Mallat, S., 2009. *A Wavelet Tour of Signal Processing: The Sparse Way*. 3rd ed. Burlington: Academic Press.
- [3] Oppenheim, A.V. and Schafer, R.W., 2010. *Discrete-Time Signal Processing*. 3rd ed. Upper Saddle River, NJ: Pearson Higher Education.
- [4] Pan, J. and Tompkins, W.J., 1985. A real-time QRS detection algorithm. *IEEE Transactions on Biomedical Engineering*, (3), pp.230-236.
- [5] Springer, D.B., Tarassenko, L. and Clifford, G.D., 2016. Logistic regression-HSMM-based heart sound segmentation. *IEEE Transactions on Biomedical Engineering*, 63(4), pp.822-832.
- [6] Torrence, C. and Compo, G.P., 1998. A practical guide to wavelet analysis. *Bulletin of the American Meteorological Society*, 79(1), pp.61-78.